## Question

Tell me about a hard technical problem you've worked on.

## Answer

One of the hardest technical problems I worked on was figuring out how to build a useful AI system for my portfolio without access to reliable training data. I'd tried building a couple of AI projects before—a fruit recognition system and a weather prediction system—and both failed because of fundamental data problems. The fruit recognition model hit 86% validation accuracy on the benchmark dataset but had literally 0% accuracy when I tested it on real photos I took myself. That's when I realised that for solo personal projects, data is almost always the insurmountable barrier: you either rely on public datasets that don't generalise, or you spend months gathering and cleaning data yourself before you can even start.

So when I built my next project, an activity recommendation system, I designed it to solve the data problem from the beginning. The system generates its own training data through use. Every time I interact with it—input my context using sliders for mood, energy, time, weather—and then pick an activity I like, that's a labeled training example the AI can learn from. I built it with a two-layer learning architecture: a Session AI that adapts quickly within the current session so it feels responsive, and a Base AI that learns slowly from all historical usage to stay stable and avoid chasing noise.

The result is a deployed system that learns from actual usage without me ever needing to source or clean an external dataset. For me, the big lesson was that the hardest AI problems aren't usually about model architecture—they're about how you get quality data in the first place. That insight now shapes how I approach any AI project: I start by asking whether the data problem is solvable before I write a line of model code. If I can't answer that, the rest doesn't matter. That's the kind of thinking I'd bring to a team—making sure we're solving the right problem before we invest heavily in a solution.

## Concepts to memorise

1. The hard problem: data acquisition
    - Trying to build useful AI systems for your portfolio, but don't have access to reliable training data.
    - Previous projects failed due to data: fruit recognition hit 86% validation accuracy but 0% on real photos; weather prediction had fundamental data coverage gaps.
    - For solo projects, data is the insurmountable barrier: public datasets don't generalise, or you spend months gathering/cleaning data.
2. The solution: self-generating training data
    - Designed the activity recommendation system to generate its own training data

through use.
    - Every interaction (input context + pick activity) is a labeled training example the AI learns from.
    - System learns progressively as you use it, no external dataset needed.
3. Two-layer architecture for balance
    - Session AI: learns quickly within the current session; feels responsive.
    - Base AI: learns slowly from all historical usage; stays stable, avoids chasing noise.
4. Result and key lesson
    - System is deployed and learns from actual usage without needing any external dataset.
    - Never needed to source or clean external training data.
    - Key insight: hardest AI problems are usually about getting quality data, not model architecture.
    - Now you start every AI project by asking whether the data problem is solvable before writing model code.
    - That strategic thinking is what you'd bring to a team—making sure you solve the right problem before investing heavily.